

Deep Learning

Module 3: Stochastic Gradient Descent and Optimizers

Quick Reference Guide

Learning Outcomes:

1. Describe incremental updates (i.e., stochastic gradient descent) to learn network parameters.
2. Explain the necessary conditions for stochastic gradient descent to converge.
3. Describe the relationship of learning rate and the ability of gradient descent to converge to a global minimum of a function.
4. Describe methods to mitigate variance problems with training using stochastic gradient descent (minibatch descent).
5. Identify the resulting effect of using a moving average on gradient trajectory (momentum).
6. Describe higher order methods: RMSProp.
7. Introduce ADAM and summarize improvements to gradient descent in terms of their effects on convergence.
8. Explain covariate shifts and batch normalization.
9. Explain backpropagation in batch normalization.
10. Explain the impact of data underspecification and the need for regularization.
11. Describe dropout as a means of regularization.
12. Identify additional heuristics and describe overall problem setup.
13. Implement a neural network to classify the phonemes of an audio file.

Gradient Descent

Given certain training instances of input–output pairs, we estimate the entire function.

In gradient descent, we:

1. Start with an initial function.
2. Adjust the network parameters to reduce the average training error at all the points. The updated parameters give a new function.
3. Repeat this iteratively until the function is arbitrarily close to the target function at the training points.

Here, before each update step, we must process the entire training set, which can be difficult and cumbersome for a large training set. The process can be simplified by making incremental updates.

In incremental updates, we:

1. Adjust the network parameters for one training point at a time.
2. Keep the updates small.
3. Process every training instance to adjust the entire function.

Incremental updates are used in stochastic gradient descent.

Stochastic Gradient Descent

If incremental updates are made in sequential order, the algorithm can show a cyclic behavior. However, if the updates are made in random order, the cyclic behavior can be prevented, and the algorithm may converge. Therefore, stochastic gradient descent (SGD) uses incremental updates, where training instances are randomly selected.

Learning Rate for Stochastic Gradient Descent

Let us define the learning rate for stochastic gradient descent.

$$w_{k+1} = w_k - \eta \frac{dL(x_i, w_i)}{dw}, \text{ where } \eta \text{ is the learning rate}$$

Naïve incremental updates can result in both a nonconvergent behavior and a never-ending cycle of updates. So, apart from the randomized presentation of inputs, the learning rate η must also shrink.

Here is the pseudocode for stochastic gradient descent.

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K; j = 0$
- Do:
 - Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
 - For all $t = 1 : T$
 - $j = j + 1$
 - For every layer k :
 - Compute $\nabla_{W_k} \text{InstanceLoss}(Y_t, d_t)$
 - Update $W_k = W_k - \eta_j \nabla_{W_k} \text{InstanceLoss}(Y_t, d_t)^T$
- Until Loss has converged

Minibatch Descent

Stochastic gradient descent uses the gradient from one sample at a time, leading to large variance between runs. Minibatch updates can be used to mitigate this variance problems.

In minibatch updates, we:

1. Adjust the function at a small randomly chosen subset of points.
2. Vary the subsets randomly at different passes.
3. Keep the adjustments small.

Here is the pseudocode for minibatch descent.

Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$

Initialize all weights $W_1, W_2, \dots, W_K; j = 0$

Do:

- Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- For all $t = 1 : b : T$
 - $j = j + 1$
 - For every layer k :
 - $\Delta W_k = 0$
 - For all $t' = t : t + b - 1$
 - For every layer k :
 - Compute $\nabla_{W_k} \text{InstanceLoss}(Y_{t'}, d_{t'})$
 - $\Delta W_k = \Delta W_k + \frac{1}{b} \nabla_{W_k} \text{InstanceLoss}(Y_{t'}, d_{t'})^T$
 - Update
 - For every layer k :
 - $W_k = W_k - \eta_j \Delta W_k$

Until Loss has converged

Momentum

Using running averages of the updates instead of the actual updates reduces oscillations and produces better results. This is called the momentum method.

$$\Delta W^{(k)} = \beta W^{(k-1)} + \eta \nabla_{W_k} \text{Loss}(W^{(k-1)})^T$$

$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

Here is the pseudocode for the momentum method.

Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$

Initialize all weights $W_1, W_2, \dots, W_K; j = 0, \Delta W_k = 0$

Do:

- Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- For all $t = 1 : b : T$
 - $j = j + 1$
 - For every layer k :
 - $\nabla_{W_k} \text{MinibatchLoss} = 0$
 - For all $t' = t : t + b - 1$
 - For every layer k :
 - Compute $\nabla_{W_k} \text{InstanceLoss}(Y_{t'}, d_{t'})$
 - $\nabla_{W_k} \text{MinibatchLoss} += \frac{1}{b} \nabla_{W_k} \text{InstanceLoss}(Y_{t'}, d_{t'})^T$
 - Update
 - For every layer k :
 - $\nabla W_k = \beta \Delta W_k - \eta_j (\nabla_{W_k} \text{MinibatchLoss})^T$

$$\triangleright W_k = W_k + \Delta W_k$$

Until Loss has converged

Higher-Order Trend Methods: RMSProp

RMSProp:

- Adjusts the learning rate for each component by the total variation of the derivative to reduce oscillation
- Is a variant of the basic minibatch SGD algorithm
- Maintains a running estimate of the mean squared value of derivatives for each parameter
 $E[(\partial_w D)^2]_k = \gamma E[(\partial_w D)^2]_{k-1} + (1 - \gamma)(\partial_w D)_k^2$, where $0 \leq \gamma \leq 1$
- Scales the learning rate by the inverse of the RMS value

$$w_{k+1} = w_k - \frac{\eta}{\sqrt{E[(\partial_w D)^2]_k + \epsilon}} \partial_w D$$

Trend Methods: RMSProp with Momentum (ADAM)

ADAM:

- Uses the smoothed running-average derivative along with the adjusted learning rate from RMSprop
- Improves convergence using smoothed updates and adaptive learning rates
- For each parameter, ADAM maintains running averages for the mean derivative, m_k , and for the mean-square derivative, v_k .

$$m_k = \delta m_{k-1} + (1 - \delta)(\partial_w D)_k; v_k = \gamma v_{k-1} + (1 - \gamma)(\partial_w D)_k^2, \text{ where } 0 \leq \delta, \gamma \leq 1$$

$$\hat{m}_k = \frac{m_k}{1 - \delta^k}; \hat{v}_k = \frac{v_k}{1 - \gamma^k}$$

$$w_{k+1} = w_k - \frac{\eta}{\sqrt{\hat{v}_k + \epsilon}} \hat{m}_k$$

Batch Normalization

In minibatch descent, each minibatch has a different distribution, a covariate shift. These shifts can be large and can affect training badly. To mitigate this, we move all the minibatches to a standard location, that is, to have a mean of zero and a standard deviation of 1. This process of normalizing the values of minibatches is called batch normalization.

Batch normalization is a covariate adjustment unit that happens after the weighted addition of inputs but before the application of activation. To simplify computation, it is done independently for each unit.

Batch Normalization Training

Batch normalization training requires:

- A forward pass that incorporates batch normalization
- A backward pass that computes the derivatives for learning the parameters

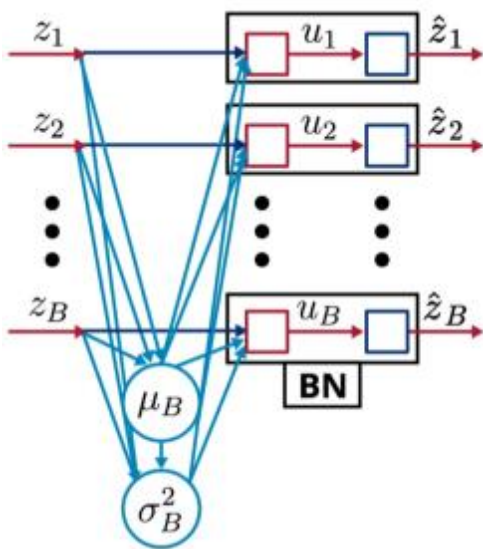


Figure 1: A section of a neural network diagram illustrating batch normalization performed using the mean μ_B and variance σ_B^2 of the batches to compute revised affine values $\hat{z}_i$ that will be fed into the activation map

Here are the values computed in the forward pass.

$$\mu_B = \frac{1}{B} \sum_{i=1}^B z_i$$

$$\sigma_B^2 = \frac{1}{B} \sum_{i=1}^B (z_i - \mu_B)^2$$

$$u_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$\hat{z}_i = \gamma u_i + \beta$$

We compute the following derivative in the backward pass.

$$\frac{d\text{Loss}}{d\hat{z}_i}$$

$$\frac{d\text{Loss}}{d\beta} = \sum_{i=1}^B \frac{d\text{Loss}}{d\hat{z}_i}$$

$$\frac{d\text{Loss}}{d\gamma} = u_i \sum_{i=1}^B \frac{d\text{Loss}}{d\hat{z}_i}$$

$$\frac{d\text{Loss}}{du_i} = \gamma \frac{d\text{Loss}}{d\hat{z}_i}$$

$$\frac{d\text{Loss}}{dz_i} = \sum_j \frac{d\text{Loss}}{du_j} \frac{du_j}{dz_i}$$

$$\frac{du_j}{dz_i} = \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} \frac{-1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{-(z_i - \mu_B)2}{B(\sigma_B^2 + \epsilon)^{3/2}} \text{ if } j = i$$

$$\frac{du_j}{dz_i} = \frac{-1}{B\sqrt{\sigma_B^2 + \epsilon}} + \frac{-(z_i - \mu_B)(z_j - \mu_B)}{B(\sigma_B^2 + \epsilon)^{3/2}} \text{ if } j \neq i$$

The complete derivative of the minibatch loss is:

$$\frac{d\text{Loss}}{dz_i} = \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} \frac{d\text{Loss}}{du_i} - \frac{1}{B\sqrt{\sigma_B^2 + \epsilon}} \sum_i \frac{d\text{Loss}}{du_j} - \frac{(z_i - \mu_B)}{B(\sigma_B^2 + \epsilon)^{3/2}} \sum_j \frac{d\text{Loss}}{du_j} (z_j - \mu_B)$$

Function Underspecification

We try to learn an entire function from just a few of its input-output relationships. However, the function maybe underspecified and there is no assurance that the network will learn what we want it to learn. So, we need additional constraints, or restrictions. For example, not constraining a function may lead to the problem of overfitting.

We can address function underspecification by introducing smoothness through weight manipulation, or regularization. In regularization, we minimize the loss while also minimizing the weights.

Regularized training:

$$L_{\text{reg}}(W) = \text{Loss}(W) + \frac{1}{2} \lambda \|W\|_F^2, \text{ where } \lambda \text{ is the regularization parameter}$$

$$\|W\|_F^2 = \text{Sum of squares of all weights and biases in every layer}$$

Minimizing divergence, keeping the weights small:

$$\hat{W} = \underset{W}{\text{argmin}} L_{\text{reg}}(W)$$

Dropout

Regularizing the network by shrinking the network parameters can mitigate data underspecification. Another influential regularization method is dropout. It is a training method where, in each iteration, for each input, we randomly turn off or dropout neurons in

the network with probability $1 - \alpha$ and retain the neurons with probability α . We don't explicitly remove the neurons. Instead, we set their outputs to 0, so they no longer contribute to the computations. During training, gradient descent is only performed over this reduced network.

Heuristics

Let us look at some tricks to optimize networks.

Early stopping:

Continued training can result in overfitting to training data. Early stopping is one approach to prevent overfitting. It involves:

- Tracking performance on a held-out validation set
- Stopping training when the performance begins to degrade

Gradient clipping:

The derivative of the loss can often be too high. Gradient clipping sets a ceiling on the derivatives. If $\partial_w D > \theta$, then $\partial_w D = \theta$.

Other tricks are:

- Data augmentation with distortions if training data is not enough
- Normalizing the training data to have a zero mean and unit variance
- Using initialization methods such as Xavier and Kaiming

The overall problem set up:

- Obtain training data.
- Choose:
 - A network architecture
 - An appropriate divergence function
 - Heuristics
 - An optimization algorithm
- Perform a grid search for hyperparameters on held-out data.
- Train and evaluate validation periodically for early stopping.